

# ARHIAX AgentCreator

*Fábrica de Agentes de Inteligencia Artificial Gobernados*

Documento de Características del Sistema

**Sinergia Consulting Group**

Estándar ARHIAX — Gobernanza por Diseño

# 1. Resumen Ejecutivo

---

ARHIAX AgentCreator es un sistema de fabricación de agentes de inteligencia artificial que nacen gobernados desde su primer instante de existencia. A diferencia de los enfoques tradicionales donde la supervisión se añade como capa posterior, ARHIAX invierte ese orden: la gobernanza es parte estructural del agente, no un accesorio.

El sistema implementa el principio de Gobernanza por Diseño mediante ocho servicios interconectados, un kit de desarrollo (SDK) en Python y diecinueve (19) módulos de políticas en lenguaje Rego evaluados por el motor Open Policy Agent (OPA).

**Características fundamentales del sistema:**

- Identidad criptográfica verificable para cada agente creado
- Niveles de autonomía certificados (escala A0 a A4) con umbrales de desviación estadística
- Evaluación de políticas antes de cada acción mediante motor OPA deny-by-default
- Registro inmutable de cada decisión con cadena HMAC-SHA256 (Evidence Store)
- Apertura automática de tickets de aprobación humana para acciones de alto impacto
- Detección de desviación conductual con escalamiento automático cuando se supera el umbral sigma del nivel
- Seis outcomes de gobernanza diferenciados que evitan bloqueos innecesarios
- Circuit breaker en el cliente del Gateway para protección ante fallos en cascada

| Componente                          | Cantidad              |
|-------------------------------------|-----------------------|
| Microservicios                      | 8 (incluyendo OPA)    |
| Módulos de políticas Rego (bundles) | 19                    |
| Endpoints de API                    | 38+                   |
| Niveles de autonomía                | 5 (A0-A4)             |
| Puertas de promoción                | 5 (G1-G5)             |
| Outcomes de gobernanza              | 6                     |
| Archivos totales del proyecto       | 252                   |
| Lenguaje principal                  | Python 3.12 + FastAPI |

## 2. Arquitectura del Sistema

---

El sistema está organizado en dos capas funcionales que operan de forma complementaria:

### 2.1 Capa de Creación

Responsable de aprovisionar nuevos agentes. Orquestada por el Creator API, esta capa registra la identidad del agente (AIM Service), inicializa su nivel de autonomía en A0 (AUT Service) y genera el código de arranque listo para usar con el SDK.

### 2.2 Capa de Operación

Responsable de gobernar cada acción del agente en tiempo real. El Gateway actúa como Policy Enforcement Point (PEP): toda acción que el agente intente ejecutar pasa obligatoriamente por el Gateway antes de realizarse. El Gateway consulta OPA, registra en el Evidence Store y devuelve el resultado en menos de 30 milisegundos.

### 2.3 Servicios del Stack

| Servicio       | Puerto | Rol Principal                                                |
|----------------|--------|--------------------------------------------------------------|
| Creator API    | 8300   | Fábrica de agentes — registro, credencial, bootstrap code    |
| Gateway (PEP)  | 8080   | Policy Enforcement Point — evalúa cada acción con OPA        |
| AIM Service    | 8200   | Agent Identity Management — credenciales y ciclo de vida     |
| AUT Service    | 8201   | Autonomy Management — niveles A0-A4, promoción y degradación |
| BBR Service    | 8202   | Behavioral Baseline Registry — desviación sigma por agente   |
| HIC Service    | 8203   | Human-in-the-Loop — tickets de aprobación con SLA            |
| Evidence Store | 8090   | Ledger inmutable JSONL con cadena HMAC-SHA256                |
| OPA Engine     | 8181   | Motor de políticas con 19 bundles Rego                       |

### 2.4 Flujo de Creación de un Agente

Cuando un desarrollador llama a POST /v1/agents/create en el Creator API, se ejecuta el siguiente flujo en orden estricto:

- Paso 1 — Validación: El Creator API valida el AgentSpec (nombre, departamento, herramientas permitidas, operaciones autorizadas).
- Paso 2 — Identidad: Se llama al AIM Service para registrar al agente y emitir su credencial de diez campos con cadena HMAC encadenada.
- Paso 3 — Autonomía: Se llama al AUT Service para inicializar el agente en nivel A0 con umbral sigma de 1.5.

- Paso 4 — Bootstrap: El Creator API genera el código Python listo para ejecutar usando el SDK ARHIAX.
- Paso 5 — Respuesta: Se devuelve el objeto GovernedAgent con agent\_id, credencial completa, código de arranque y URL del Gateway.

## 2.5 Flujo de Gobernanza de una Acción

En tiempo de operación, cuando el agente intenta ejecutar cualquier acción decorada con @governed\_tool, el SDK ejecuta automáticamente el siguiente flujo:

- Paso 1 — El SDK llama a POST /v1/decide en el Gateway con el subject, action, resource y context de la acción.
- Paso 2 — El Gateway verifica que el body no supere 1 MiB (protección anti-DoS).
- Paso 3 — El Gateway detecta patrones de inyección de forma local antes de consultar OPA.
- Paso 4 — El Gateway consulta OPA en POST /v1/data/arhiar/main y recibe la decisión de política.
- Paso 5 — El Gateway registra la decisión en el Evidence Store (POST /v1/evidence).
- Paso 6 — El Gateway devuelve {allow, reasons, obligations, evidence\_id} al SDK.
- Paso 7 — El SDK interpreta el outcome y ejecuta, bloquea o escala según corresponda.
- Paso 8 — El SDK registra la observación en el BBR Service de forma asíncrona (fire-and-forget).

# 3. Descripción Detallada de Servicios

## 3.1 Creator API — Fábrica de Agentes Gobernados

El Creator API es el punto de entrada principal para la creación de agentes. Orquesta el registro en AIM, la inicialización en AUT y la generación del código de arranque.

| Endpoint                            | Método | Descripción                                         |
|-------------------------------------|--------|-----------------------------------------------------|
| POST /v1/agents/create              | POST   | Crea un agente gobernado completo                   |
| GET /v1/agents                      | GET    | Lista todos los agentes del sistema                 |
| GET /v1/agents/{agent_id}           | GET    | Detalles completos de un agente                     |
| POST /v1/agents/{agent_id}/evaluate | POST   | Evalúa una acción en modo test (sin ejecutar)       |
| POST /v1/agents/{agent_id}/promote  | POST   | Promueve el nivel de autonomía (requiere 5 puertas) |
| DELETE /v1/agents/{agent_id}        | DELETE | Da de baja un agente y revoca su credencial         |
| GET /healthz                        | GET    | Health check básico                                 |
| GET /readyz                         | GET    | Readiness: verifica AIM, AUT y Gateway              |

Parámetros del endpoint POST /v1/agents/create:

| Campo                     | Tipo     | Requerido | Descripción                                         |
|---------------------------|----------|-----------|-----------------------------------------------------|
| name                      | string   | Sí        | Nombre descriptivo del agente                       |
| description               | string   | No        | Descripción funcional del agente                    |
| department_id             | string   | Sí        | Unidad organizacional a la que pertenece            |
| supervisor_id             | string   | Sí        | Identificador del supervisor humano responsable     |
| authorization_boundary_id | string   | No        | Dominio de confianza (default: "default")           |
| permitted_tools           | string[] | No        | Lista de herramientas que el agente puede invocar   |
| permitted_data_scopes     | string[] | No        | Scopes de datos autorizados                         |
| permitted_operations      | string[] | No        | Tipos de operación (default: modelInvoke, toolCall) |
| rotation_days             | int      | No        | Días hasta expiración de credencial (default: 90)   |

## 3.2 AIM Service — Agent Identity Management

El AIM Service es el registro de identidad de todos los agentes. Emite credenciales con diez campos, implementa una cadena HMAC para verificar la autenticidad de cada credencial y gestiona el ciclo de vida del agente (ACTIVE, ROTATING, SUSPENDED, RETIRED).

Estructura de la credencial (10 campos):

| Campo                     | Descripción                                                                  |
|---------------------------|------------------------------------------------------------------------------|
| agent_id                  | Identificador único generado automáticamente (ej. agent-a1b2c3d4e5f6)        |
| name                      | Nombre descriptivo del agente                                                |
| supervisor_id             | ID del supervisor humano responsable                                         |
| department_id             | Unidad organizacional                                                        |
| authorization_boundary_id | Dominio de confianza y autorización                                          |
| autonomy_level            | Nivel actual A0–A4 (siempre inicia en A0)                                    |
| credential_issued_at      | Marca de tiempo de emisión en ISO 8601                                       |
| credential_expires_at     | Marca de tiempo de expiración en ISO 8601                                    |
| rotation_policy           | Política de rotación (ej. "90d")                                             |
| lifecycle_state           | Estado: ACTIVE   ROTATING   SUSPENDED   RETIRED                              |
| parent_chain_hmac         | Cadena HMAC-SHA256 de identidad (computed: agent_id:supervisor_id:issued_at) |
| permitted_tools           | Lista de herramientas autorizadas                                            |
| permitted_data_scopes     | Lista de scopes de datos autorizados                                         |
| permitted_operations      | Lista de tipos de operación autorizados                                      |

| Endpoint                                 | Método | Descripción                                         |
|------------------------------------------|--------|-----------------------------------------------------|
| POST /v1/agents/register                 | POST   | Registra agente y emite credencial inicial (A0)     |
| GET /v1/credentials/{agent_id}           | GET    | Obtiene la credencial completa del agente           |
| GET /v1/agents                           | GET    | Lista todos los agentes registrados                 |
| POST /v1/credentials/{agent_id}/rotate   | POST   | Rota la credencial manteniendo el agent_id          |
| POST /v1/credentials/{agent_id}/revoke   | POST   | Suspende la credencial (lifecycle_state: SUSPENDED) |
| POST /v1/credentials/{agent_id}/autonomy | POST   | Actualiza el autonomy_level en la credencial        |
| GET /v1/credentials/{agent_id}/history   | GET    | Historial completo de cambios de autonomía          |

3.3 AUT Service — Autonomy Management

El AUT Service gestiona la Escala de Autonomía Adaptativa A0–A4. Determina si una acción está dentro del rango permitido para el nivel del agente, evalúa las cinco puertas de promoción, registra degradaciones automáticas y mantiene el historial completo de cambios de nivel. Implementa los controles AUT-C01 a AUT-C05 del estándar ARHIAX.

Escala de autonomía:

| Nivel | Nombre      | Umbral Sigma ( $\sigma$ ) | Qué requiere aprobación humana |
|-------|-------------|---------------------------|--------------------------------|
| A0    | Inerte      | 1.5 $\sigma$              | Toda acción sin excepción      |
| A1    | Supervisado | 2.0 $\sigma$              | Acciones de alto impacto       |
| A2    | Guiado      | 2.5 $\sigma$              | Acciones de impacto medio      |
| A3    | Autónomo    | 3.0 $\sigma$              | Solo acciones críticas         |

|    |            |              |                             |
|----|------------|--------------|-----------------------------|
| A4 | Adaptativo | 3.5 $\sigma$ | Solo excepciones explícitas |
|----|------------|--------------|-----------------------------|

#### Acciones clasificadas como alto impacto (siempre generan notificación HIC):

- delete — Eliminación de datos o recursos
- transfer\_funds — Transferencia de fondos
- modify\_policy — Modificación de políticas
- promote\_agent — Promoción de otro agente
- revoke\_credential — Revocación de credenciales
- deploy — Despliegue de artefactos
- override\_safety — Anulación de controles de seguridad
- grant\_permission — Otorgamiento de permisos
- external\_api\_write — Escritura en APIs externas

| Endpoint                             | Método | Descripción                                                 |
|--------------------------------------|--------|-------------------------------------------------------------|
| GET /v1/autonomy/{agent_id}          | GET    | Nivel actual, umbral sigma y fecha de vigencia              |
| POST /v1/autonomy/check              | POST   | Evalúa si una acción está permitida para el nivel           |
| POST /v1/autonomy/{agent_id}/promote | POST   | Solicita promoción — evalúa las 5 puertas                   |
| POST /v1/autonomy/{agent_id}/degrade | POST   | Degrada un nivel (llamado cuando $\sigma > \text{umbral}$ ) |
| GET /v1/autonomy/{agent_id}/history  | GET    | Historial de eventos de autonomía                           |

### 3.4 BBR Service — Behavioral Baseline Registry

El BBR Service registra observaciones del comportamiento de cada agente (duración de operaciones, tokens consumidos, outcomes) y calcula en tiempo real cuánto se está desviando el agente de su línea base histórica. Esta desviación se expresa en unidades de desviación estándar (sigma). Implementa el control DTG-C04 del estándar ARHIAX.

#### Lógica de cálculo de sigma:

- El BBR mantiene estadísticas corrientes sobre las últimas 200 observaciones por agente.
- Calcula mean\_duration (media de duración en ms) y std\_duration (desviación estándar de duración).
- Calcula mean\_tokens (media de tokens) y std\_tokens (desviación estándar de tokens).
- $\sigma_{\text{duration}} = |\text{duration\_actual} - \text{mean\_duration}| / \text{std\_duration}$
- $\sigma_{\text{tokens}} = |\text{tokens\_actual} - \text{mean\_tokens}| / \text{std\_tokens}$
- $\sigma_{\text{final}} = \max(\sigma_{\text{duration}}, \sigma_{\text{tokens}})$
- Requiere mínimo 5 observaciones para tener línea base válida (has\_baseline: true).
- Con menos de 5 observaciones devuelve has\_baseline: false y el Gateway usa ALLOW\_WITH\_MONITORING.
- Diseño fail-open: si el servicio no está disponible, no bloquea la operación.

| Endpoint                                 | Método | Descripción                                                  |
|------------------------------------------|--------|--------------------------------------------------------------|
| POST /v1/baseline/{agent_id}/observe     | POST   | Registra una observación de comportamiento                   |
| GET /v1/baseline/{agent_id}              | GET    | Estadísticas actuales: mean, std, sample_count, has_baseline |
| POST /v1/baseline/{agent_id}/score       | POST   | Calcula sigma de desviación para una operación específica    |
| GET /v1/baseline/{agent_id}/observations | GET    | Lista las últimas N observaciones (default: 50)              |

### 3.5 HIC Service — Human-in-the-Loop Checkpoints

El HIC Service gestiona los tickets de aprobación humana que se generan cuando un agente intenta una acción de alto impacto o cuando su desviación conductual supera el umbral de su nivel de autonomía. Incluye SLA por severidad, notificaciones vía webhook configurable y registro completo de decisiones humanas. Implementa los controles HIC-C01 a HIC-C05 del estándar ARHIAX.

**SLA por severidad:**

| Severidad | Tiempo máximo de respuesta | Consecuencia si se vence el SLA                     |
|-----------|----------------------------|-----------------------------------------------------|
| CRITICAL  | 5 minutos                  | El ticket pasa automáticamente a estado SLA_EXPIRED |
| HIGH      | 15 minutos                 | El ticket pasa automáticamente a estado SLA_EXPIRED |
| MEDIUM    | 1 hora                     | El ticket pasa automáticamente a estado SLA_EXPIRED |
| LOW       | 24 horas                   | El ticket pasa automáticamente a estado SLA_EXPIRED |

**Estados posibles de un ticket:**

| Estado      | Descripción                              | Transición                    |
|-------------|------------------------------------------|-------------------------------|
| PENDING     | Ticket creado, esperando decisión humana | Inicial                       |
| APPROVED    | El supervisor humano aprobó la acción    | Desde PENDING vía approve()   |
| REJECTED    | El supervisor humano rechazó la acción   | Desde PENDING vía reject()    |
| SLA_EXPIRED | Tiempo máximo de respuesta agotado       | Desde PENDING automáticamente |

**Triggers que abren un ticket:**

| Trigger                                  | Outcome del Gateway         | Severidad típica |
|------------------------------------------|-----------------------------|------------------|
| Acción de alto impacto (checks ok)       | ALLOW_WITH_HIC_NOTIFICATION | HIGH             |
| Desviación sigma supera umbral del nivel | ESCALATE_TO_HUMAN           | CRITICAL         |
| Inyección detectada en payload           | DENY_WITH_INCIDENT          | CRITICAL         |

El servicio envía notificaciones vía webhook configurable (HIC\_WEBHOOK\_URL) compatible con Slack incoming webhooks, Microsoft Teams y cualquier endpoint HTTP. Cada notificación incluye ticket\_id, agent\_id, action, severity, reason, sla\_deadline, approve\_url y reject\_url.

| Endpoint                             | Método | Descripción                                      |
|--------------------------------------|--------|--------------------------------------------------|
| POST /v1/tickets                     | POST   | Crea ticket de aprobación humana                 |
| GET /v1/tickets/{ticket_id}          | GET    | Estado actual de un ticket                       |
| GET /v1/tickets                      | GET    | Lista tickets (filtros: agent_id, status, limit) |
| POST /v1/tickets/{ticket_id}/approve | POST   | Aprobación por el supervisor humano              |
| POST /v1/tickets/{ticket_id}/reject  | POST   | Rechazo por el supervisor humano                 |
| GET /v1/tickets/expired/check        | GET    | Verifica y marca tickets vencidos por SLA        |

### 3.6 Gateway — Policy Enforcement Point

El Gateway es el Policy Enforcement Point (PEP) del sistema. Todo agente que quiera ejecutar cualquier acción debe pedir permiso al Gateway primero. El Gateway consulta el motor OPA, registra la decisión en el Evidence Store y devuelve el resultado en menos de 30 milisegundos. Implementa los controles OPA-C01 a OPA-C04 del estándar ARHIAX.

#### Política de fallos:

| Servicio que falla         | Comportamiento del Gateway                            |
|----------------------------|-------------------------------------------------------|
| OPA no disponible          | HTTP 503 — fail-closed: no se permite la acción       |
| Evidence Store no responde | Se retorna la decisión igual (fail-open en auditoría) |
| Inyección detectada        | allow: false con reasons: ["INJECTION_DETECTED"]      |

#### Patrones de inyección detectados localmente por el Gateway:

- "ignore previous", "disregard" → Prompt injection clásica
- "<script>", "javascript:" → Cross-Site Scripting (XSS)
- "UNION SELECT", "DROP TABLE", "'; --" → SQL injection
- "{{", "}}", "\${", "\$(" → Template y shell injection
- "\x00", "\x1b", "%00", "%0a%0d" → Null byte y caracteres de control

#### Métricas Prometheus disponibles en :8080/metrics:

- arhiax\_gateway\_decide\_total{outcome="allow"} — Total de decisiones permitidas
- arhiax\_gateway\_decide\_total{outcome="deny"} — Total de decisiones denegadas
- arhiax\_gateway\_opa\_errors\_total — Errores de comunicación con OPA
- arhiax\_gateway\_evidence\_errors\_total — Errores de comunicación con Evidence Store

### 3.7 Evidence Store — Ledger Inmutable

El Evidence Store mantiene un ledger append-only en formato JSONL donde cada entrada está encadenada mediante HMAC-SHA256 con la entrada anterior. Esta cadena hace imposible modificar o eliminar un registro sin invalidar toda la cadena posterior, proporcionando un registro forense

completo e inalterable de cada decisión de gobernanza. Implementa los controles EGA-C01 a EGA-C04 del estándar ARHIAX.

#### Estructura de la cadena HMAC:

- Entrada 1: datos → HMAC(prev="0000...0000", entry\_json) = H1
- Entrada 2: datos → HMAC(prev=H1, entry\_json) = H2
- Entrada 3: datos → HMAC(prev=H2, entry\_json) = H3
- Para modificar la entrada 2 sin ser detectado, un atacante tendría que recalcular H2, H3, H4... hasta el final — imposible sin el secreto HMAC.

#### Estructura de cada registro del ledger:

| Campo           | Descripción                                             |
|-----------------|---------------------------------------------------------|
| id              | Identificador único (ej. ev-0000001234)                 |
| sequence_number | Número de secuencia incremental                         |
| timestamp       | Marca de tiempo ISO 8601 UTC                            |
| subject         | ID del agente que ejecutó la acción                     |
| action          | Tipo de acción (toolCall, modelInvoke, etc.)            |
| resource        | Recurso objetivo de la acción                           |
| context         | Contexto completo de la invocación (JSON)               |
| decision        | Resultado de la evaluación (true/false)                 |
| reasons         | Lista de razones de la decisión                         |
| obligations     | Obligaciones impuestas (rate_limit, audit_log)          |
| prev_hash       | Hash HMAC-SHA256 de la entrada anterior (cadena Merkle) |
| entry_hmac      | Hash HMAC-SHA256 de este registro                       |

#### Política de retención de evidencia (estándar ARHIAX):

| Tier   | Tipos de registro                       | Retención |
|--------|-----------------------------------------|-----------|
| Tier 1 | ATT (attestaciones), APR (aprobaciones) | 7 años    |
| Tier 2 | LOG (logs), MET (métricas)              | 3 años    |
| Tier 3 | TST (tests)                             | 1 año     |

| Endpoint                      | Método | Descripción                                    |
|-------------------------------|--------|------------------------------------------------|
| POST /v1/evidence             | POST   | Registra nueva entrada en el ledger            |
| GET /v1/evidence/{id}         | GET    | Recupera registro específico por ID            |
| GET /v1/evidence              | GET    | Lista registros recientes (?limit=20&subject=) |
| GET /v1/head                  | GET    | Estado actual: sequence, last_hash, entries    |
| GET /v1/evidence/verify/chain | GET    | Verifica integridad completa de la cadena HMAC |

## 4. Motor de Políticas OPA — 19 Bundles Rego

El sistema utiliza Open Policy Agent (OPA) 0.68.0 como motor de evaluación de políticas. Las políticas están organizadas en 19 bundles temáticos en lenguaje Rego. La arquitectura es deny-by-default: toda acción está bloqueada salvo que una regla explícita de permiso aplique.

### 4.1 Lógica de Evaluación de Políticas

El flujo de evaluación sigue este orden estricto:

- Deny-by-default: allow := false si ninguna regla de permiso aplica.
- ¿Hay inyección en el payload? → DENY (INJECTION\_DETECTED)
- ¿El tipo de operación es válido? (toolCall, modelInvoke, dataAccess, interAgentCall)
- ¿El nivel solicitado es menor o igual al nivel certificado del agente?
- ¿La acción es crítica con nivel A0–A3? → DENY
- ¿La acción es de alto impacto? → ALLOW + obligation: audit\_log
- ALLOW + obligation: rate\_limit (límite de requests por minuto)

### 4.2 Tipos de Operación Válidos

| Tipo           | Descripción                                                        |
|----------------|--------------------------------------------------------------------|
| toolCall       | Invocación de una herramienta definida en la credencial del agente |
| modelInvoke    | Llamada directa al modelo de lenguaje (LLM)                        |
| dataAccess     | Acceso a datos en el ámbito autorizado                             |
| interAgentCall | Llamada de un agente gobernado a otro agente gobernado             |

### 4.3 Tipos de Obligaciones

| Tipo       | Valor  | Significado                                        |
|------------|--------|----------------------------------------------------|
| rate_limit | int    | Máximo de requests por minuto permitidos al agente |
| audit_log  | string | Nivel de detalle de log requerido para la acción   |

## 5. Los Seis Outcomes de Gobernanza

---

Cada acción evaluada por el Gateway termina en uno de seis outcomes diferenciados. Esta gradación fina evita bloqueos innecesarios y habilita monitoreo selectivo sin sacrificar seguridad.

| Outcome                     | Significado                          | Comportamiento del SDK                          |
|-----------------------------|--------------------------------------|-------------------------------------------------|
| ALLOW                       | Todo correcto, política permite      | Ejecuta la acción normalmente                   |
| ALLOW_WITH_MONITORING       | BBR sin línea base (menos de 5 obs.) | Ejecuta + registra métricas adicionales         |
| ALLOW_WITH_HIC_NOTIFICATION | Alto impacto, checks aprobados       | Ejecuta + abre ticket HIC + notifica supervisor |
| DENY                        | Política o autonomía violada         | Lanza excepción ARHIAXDenied                    |
| DENY_WITH_INCIDENT          | Inyección de prompt detectada        | Lanza ARHIAXInjectionDetected + crea incidente  |
| ESCALATE_TO_HUMAN           | Desviación sigma excedida            | Lanza ARHIAXEscalated + abre ticket HIC         |

## 6. Las Cinco Puertas de Promoción de Autonomía

---

Para que un agente pueda subir de nivel de autonomía (por ejemplo, de A0 a A1), las cinco puertas de evaluación deben estar simultáneamente en estado verde (true). Si alguna puerta falla, la promoción se deniega y el sistema reporta cuáles puertas fallaron junto con su descripción.

| Puerta | Nombre         | Criterio de aprobación                                                                            |
|--------|----------------|---------------------------------------------------------------------------------------------------|
| G1     | G1_performance | Métricas de desempeño satisfactorias (precisión, tasa de éxito de tareas)                         |
| G2     | G2_security    | Sin incidentes de seguridad en la ventana de evaluación de 30 días (0 inyecciones, 0 violaciones) |
| G3     | G3_business    | Aprobación formal de la unidad de negocio (stakeholder sign-off documentado)                      |
| G4     | G4_history     | Historial limpio: sin degradaciones de autonomía en los últimos 30 días                           |
| G5     | G5_governance  | Revisión del equipo de gobernanza aprobada (audit committee sign-off)                             |

La degradación de autonomía ocurre automáticamente cuando el sigma observado supera el umbral del nivel actual del agente. El AUT Service registra el evento y puede bajar el nivel si se supera el umbral más de tres veces en una ventana de tiempo. La degradación automática bloquea la promoción futura hasta que la puerta G4 vuelva a estar en verde (30 días sin degradaciones).

## 7. SDK Python — arhiac-sdk

El SDK Python es la biblioteca que los desarrolladores instalan en sus proyectos de agentes para conectar automáticamente con la infraestructura de gobernanza ARHIAX. El SDK gestiona toda la comunicación con el Gateway, el AIM Service, el HIC Service y el BBR Service de forma transparente para el desarrollador.

### 7.1 Instalación

```
pip install -e sdk/python/
```

### 7.2 ARHIAXAgent — Clase Base

ARHIAXAgent es la clase base de la que deben heredar todos los agentes gobernados. Provee el context manager asíncrono (async with) que carga automáticamente la credencial del agente al iniciar y limpia el estado al finalizar.

### 7.3 El Decorador @governed\_tool

El decorador @governed\_tool es el mecanismo central del SDK. Envuelve automáticamente cualquier método Python con la lógica de gobernanza ARHIAX. El desarrollador no necesita escribir ningún código de gobernanza explícito.

El decorador ejecuta automáticamente:

- Verificación de que la herramienta está en la lista de herramientas permitidas de la credencial.
- Llamada al Gateway con action, resource y context.
- Interpretación del outcome recibido.
- Si ALLOW: ejecuta la función original.
- Si ALLOW\_WITH\_HIC\_NOTIFICATION: ejecuta + envía notificación HIC.
- Si DENY: lanza ARHIAXDenied sin ejecutar la función.
- Si DENY\_WITH\_INCIDENT: lanza ARHIAXInjectionDetected.
- Si ESCALATE\_TO\_HUMAN: lanza ARHIAXEscalated.
- Registro asíncrono de la observación en BBR (siempre, independiente del outcome).

### 7.4 Excepciones del SDK

| Excepción                | Cuándo se lanza                                           | Atributos clave                        |
|--------------------------|-----------------------------------------------------------|----------------------------------------|
| ARHIAXDenied             | Cuando el Gateway devuelve DENY                           | action, resource, reasons, evidence_id |
| ARHIAXEscalated          | Cuando la desviación sigma supera el umbral del nivel     | action, resource, ticket_id            |
| ARHIAXInjectionDetected  | Cuando se detecta inyección de prompt en el payload       | evidence_id                            |
| ARHIAXCredentialExpired  | Cuando la credencial del agente está vencida o suspendida | agent_id, lifecycle_state              |
| ARHIAXToolNotPermitted   | Cuando la herramienta no está en la lista de permitidas   | agent_id, tool_name, permitted         |
| ARHIAXServiceUnavailable | Cuando un servicio ARHIAX no responde                     | service, detail                        |

## 7.5 GatewayClient — Circuit Breaker

El GatewayClient implementa un circuit breaker para proteger al agente ante fallos en cascada del Gateway. El circuit breaker tiene tres estados:

- CLOSED (normal): las solicitudes pasan al Gateway normalmente.
- OPEN (fallo): después de 5 fallos consecutivos, el circuito se abre y todas las solicitudes fallan inmediatamente con ARHIAXServiceUnavailable sin intentar contactar al Gateway.
- HALF\_OPEN (recuperación): después de 30 segundos en estado OPEN, el circuito permite una solicitud de prueba. Si tiene éxito, vuelve a CLOSED. Si falla, vuelve a OPEN.

Adicionalmente, el GatewayClient implementa reintentos con retroceso exponencial para manejar fallos transitorios de red.

## 7.6 Ejemplo Completo de Uso

```
from arhiax import ARHIAXAgent, governed_tool
from arhiax import ARHIAXDenied, ARHIAXEscalated, ARHIAXInjectionDetected

class AgenteDeAnálisis(ARHIAXAgent):
    agent_id      = "agent-abc123"          # Obtenido del Creator API
    gateway_url   = "http://localhost:8080"
    aim_url       = "http://localhost:8200"

    @governed_tool(resource="consultar_base_datos", severity="MEDIUM")
    async def consultar_base_datos(self, query: str) -> dict:
        return await mi_db.execute(query)

    @governed_tool(resource="enviar_email", severity="HIGH")
    async def enviar_email(self, to: str, body: str) -> bool:
        return await email_service.send(to, body)

    async def run(self, tarea: str) -> str:
        return await self.invoke_model(prompt=tarea)

async def main():
    async with AgenteDeAnálisis() as agente:
        try:
            resultado = await agente.consultar_base_datos(
                "SELECT * FROM clientes"
            )
            print(resultado)
        except ARHIAXDenied as e:
            print(f"Bloqueado por política: {e.reasons}")
        except ARHIAXEscalated as e:
            print(f"Escalado a humano - Ticket: {e.ticket_id}")
        except ARHIAXInjectionDetected as e:
            print(f"Inyección detectada - Evidencia: {e.evidence_id}")
```

# 8. Modelo de Seguridad

ARHIAX AgentCreator implementa defensa en profundidad con cinco capas de seguridad independientes que operan en conjunto.

## 8.1 Capas de Seguridad

### Capa 1 — Identidad (AIM Service):

- Cada agente tiene credencial con HMAC-SHA256 encadenado.
- Las credenciales expiran automáticamente (default: 90 días).
- Los ciclos de vida SUSPENDED y RETIRED bloquean operación inmediata.
- El secreto HMAC nunca viaja en requests: solo se usa localmente para verificación.

### Capa 2 — Políticas (OPA/Rego):

- Deny-by-default: toda acción está bloqueada salvo regla explícita de permiso.
- 19 bundles de políticas ARHIAX.
- Si OPA no responde → fail-closed (DENY automático, HTTP 503).
- Hot-reload de políticas sin necesidad de reiniciar servicios.

### Capa 3 — Detección de amenazas (Gateway):

- Detección local de 13 patrones de inyección antes de llegar a OPA.
- Límite de 1 MiB por request (protección anti-DoS básica).
- Outcomes diferenciados por tipo de violación.

### Capa 4 — Auditoría (Evidence Store):

- Ledger JSONL append-only con cadena HMAC-SHA256.
- Imposible modificar un registro sin invalidar toda la cadena posterior.
- Verificación de integridad bajo demanda con GET /v1/evidence/verify/chain.

### Capa 5 — Supervisión humana (HIC Service):

- Acciones de alto impacto generan notificación humana obligatoria.
- SLA por severidad con expiración automática si no hay respuesta.
- Toda decisión humana queda registrada con reviewer\_id.

## 8.2 Variables de Entorno — Secretos Críticos

| Variable             | Descripción                                    | Riesgo si no se cambia en producción                              |
|----------------------|------------------------------------------------|-------------------------------------------------------------------|
| AIM_HMAC_SECRET      | Secreto para la cadena HMAC de credenciales    | Un atacante podría forjar credenciales HMAC válidas               |
| EVIDENCE_HMAC_SECRET | Secreto para la cadena HMAC del ledger         | Un atacante podría forjar entradas en el ledger sin detección     |
| HIC_WEBHOOK_URL      | URL webhook para notificaciones de tickets HIC | Los supervisores no recibirán notificaciones de acciones críticas |
| ANTHROPIC_API_KEY    | API key del modelo de lenguaje                 | El agente no podrá invocar el modelo LLM                          |

### 8.3 Checklist de Seguridad Pre-Producción

- Cambiar AIM\_HMAC\_SECRET por valor generado aleatoriamente (`python -c "import secrets; print(secrets.token_hex(32))"`)
- Cambiar EVIDENCE\_HMAC\_SECRET por valor generado aleatoriamente
- Configurar HIC\_WEBHOOK\_URL para notificaciones de tickets críticos
- Verificar que OPA y Evidence Store NO tienen puertos expuestos al exterior
- Configurar backups automáticos de volúmenes Docker
- Configurar monitoreo de /healthz y /readyz de todos los servicios
- Ejecutar GET /v1/evidence/verify/chain en cron de integridad (recomendado: cada hora)
- Revisar y ajustar las políticas OPA en runtime/bundles/ para el caso de uso específico
- Establecer proceso de rotación de credenciales de agentes cada 90 días
- Documentar quiénes son los supervisor\_id y sus responsabilidades

## 9. Despliegue y Operación

### 9.1 Inicio Rápido con Docker Compose

```
# 1. Clonar y configurar variables de entorno
cp .env.example .env
# Editar .env con los secretos correspondientes

# 2. Levantar el stack completo
docker compose up -d

# 3. Verificar estado de todos los servicios
docker compose ps

# 4. Ver logs en tiempo real
docker compose logs -f gateway

# 5. Crear el primer agente gobernado
curl -X POST http://localhost:8300/v1/agents/create \
  -H "Content-Type: application/json" \
  -d '{"name": "MiAgente", "department_id": "dept-ops",
      "supervisor_id": "supervisor-001",
      "permitted_tools": ["consultar_datos"]}'

# 6. Instalar el SDK Python
pip install -e sdk/python/

# 7. Ejecutar el ejemplo incluido
python sdk/python/examples/agente_analista.py
```

### 9.2 Variables de Entorno del Sistema

| Variable               | Servicio       | Default                    | Descripción                                 |
|------------------------|----------------|----------------------------|---------------------------------------------|
| OPA_URL                | Gateway        | http://opa:8181            | URL del motor OPA                           |
| EVIDENCE_STORE_URL     | Gateway        | http://evidence-store:8090 | URL del Evidence Store                      |
| MAX_REQUEST_BODY_BYTES | Gateway        | 1048576                    | Límite de body (1 MiB)                      |
| LEDGER_PATH            | Evidence Store | /data/evidence.jsonl       | Ruta al archivo JSONL del ledger            |
| EVIDENCE_HMAC_SECRET   | Evidence Store | (cambiar en producción)    | Secreto para cadena HMAC del ledger         |
| AIM_HMAC_SECRET        | AIM Service    | (cambiar en producción)    | Secreto para cadena HMAC de credenciales    |
| AUT_DB_PATH            | AUT Service    | /data/aut.db               | Ruta a la base de datos SQLite de autonomía |
| BBR_DB_PATH            | BBR Service    | /data/bbr.db               | Ruta a la base de datos SQLite de baseline  |
| HIC_DB_PATH            | HIC Service    | /data/hic.db               | Ruta a la base de datos SQLite de tickets   |
| HIC_WEBHOOK_URL        | HIC Service    | ""                         | URL webhook para notificaciones (opcional)  |

|                    |             |                       |                                     |
|--------------------|-------------|-----------------------|-------------------------------------|
| ANTHROPIC_API_KEY  | SDK/Agentes | (requerido)           | API key del modelo de lenguaje      |
| ARHIAX_GATEWAY_URL | SDK/Agentes | http://localhost:8080 | URL del Gateway desde el agente     |
| ARHIAX_AIM_URL     | SDK/Agentes | http://localhost:8200 | URL del AIM Service desde el agente |
| ARHIAX_HIC_URL     | SDK/Agentes | http://localhost:8203 | URL del HIC Service desde el agente |
| ARHIAX_BBR_URL     | SDK/Agentes | http://localhost:8202 | URL del BBR Service desde el agente |

### 9.3 Requisitos del Sistema

| Componente     | Versión mínima | Notas                           |
|----------------|----------------|---------------------------------|
| Docker Desktop | 4.x            | o Docker Engine en Linux        |
| Docker Compose | v2             | Incluido en Docker Desktop 4.x  |
| Python         | 3.10+          | Para el SDK y desarrollo        |
| RAM            | 2 GB           | Para el stack completo en local |
| Almacenamiento | 1 GB           | Para volúmenes de datos y logs  |

### 9.4 Helm Charts para Kubernetes

El proyecto incluye Helm charts completos para despliegue en Kubernetes en el directorio chart/. Los charts incluyen:

- Deployments para todos los microservicios Python
- StatefulSet para el Evidence Store (almacenamiento persistente)
- Horizontal Pod Autoscaler (HPA) para Gateway y OPA
- Pod Disruption Budget (PDB) para alta disponibilidad
- Network Policies para aislamiento de red
- ServiceMonitor para integración con Prometheus
- RBAC y ServiceAccounts
- ConfigMaps con los 19 bundles de políticas Rego
- Values files para múltiples entornos (desarrollo, staging, producción)

## 10. Filosofía de Diseño

| Principio                    | Implementación en ARHIAX AgentCreator                                                                                                      |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Gobernanza por Diseño        | Los agentes nacen gobernados. La gobernanza no es un módulo adicional sino la estructura central del agente.                               |
| Deny-by-Default              | OPA niega toda acción salvo que una regla explícita de permiso aplique. El silencio de la política es una denegación.                      |
| Fail-Closed en Autenticación | Si OPA o AIM no responden, la respuesta es siempre DENY. No se permite operar sin gobernanza.                                              |
| Fail-Open en Auditoría       | Si Evidence Store falla, la decisión de gobernanza se retorna igual. El registro no bloquea la operación.                                  |
| 6 Outcomes, No 2             | La gradación fina (ALLOW, ALLOW_WITH_MONITORING, ALLOW_WITH_HIC, DENY, DENY_WITH_INCIDENT, ESCALATE_TO_HUMAN) evita bloqueos innecesarios. |
| Evidencia Primero            | Toda decisión produce registro inmutable antes de retornar. No hay decisión de gobernanza sin evidencia.                                   |
| Autonomía Escalonada         | Sin saltos de nivel. La promoción es siempre incremental: A0→A1→A2→A3→A4. Las 5 puertas deben estar verdes simultáneamente.                |
| Circuit Breaker              | El GatewayClient abre circuito tras 5 fallos consecutivos para evitar cascadas. Recuperación automática en 30 segundos.                    |
| BBR Fail-Open                | El Behavioral Baseline Registry es fail-open. Si no está disponible, el agente opera con ALLOW_WITH_MONITORING.                            |
| HIC con SLA                  | Los tickets de aprobación humana tienen tiempo máximo de respuesta (5 min a 24 horas según severidad). El vencimiento es explícito.        |

## 11. Glosario de Términos ARHIAX

| Término            | Significado                                                                    |
|--------------------|--------------------------------------------------------------------------------|
| ARHIAX             | Nombre del sistema de gobernanza de agentes IA de Sinergia Consulting Group    |
| AIM                | Agent Identity Management — módulo de gestión de identidad y credenciales      |
| ATK                | Agent Trust Kernel — envoltura de 5 verificaciones de confianza                |
| AUT                | Autonomy Management — gestión de la escala de autonomía A0–A4                  |
| BBR                | Behavioral Baseline Registry — registro de línea base conductual               |
| Circuit Breaker    | Patrón de resiliencia: corta el circuito tras N fallos para evitar cascadas    |
| Credential         | Objeto de 10 campos que identifica criptográficamente a un agente              |
| DTG                | Detection & Telemetry Gateway — telemetría y detección de anomalías            |
| EGA                | Evidence & Governance Audit — auditoría y evidencia de gobernanza              |
| Evidence Store     | Ledger JSONL append-only con cadena HMAC-SHA256                                |
| Fail-Closed        | Ante fallo, denegar por defecto (más seguro)                                   |
| Fail-Open          | Ante fallo, permitir por defecto (menos disruptivo)                            |
| Gateway            | Policy Enforcement Point — intermediario entre agente y ejecución              |
| HIC                | Human-in-the-Loop Checkpoints — sistema de aprobación humana con SLA           |
| HMAC               | Hash-based Message Authentication Code — código de autenticación criptográfico |
| Lifecycle State    | Estado del ciclo de vida: ACTIVE, ROTATING, SUSPENDED, RETIRED                 |
| OPA                | Open Policy Agent — motor de evaluación de políticas en lenguaje Rego          |
| Outcome            | Resultado de gobernanza: uno de 6 valores posibles                             |
| PEP                | Policy Enforcement Point — punto de cumplimiento de políticas                  |
| Rego               | Lenguaje declarativo de OPA para escribir políticas de acceso                  |
| Sigma ( $\sigma$ ) | Desviación estándar — unidad de medida de desviación conductual en BBR         |
| SLA                | Service Level Agreement — tiempo máximo de respuesta para tickets HIC          |
| SDK                | Software Development Kit — biblioteca Python para crear agentes gobernados     |

## 12. Estructura del Proyecto

El proyecto ARHIAX AgentCreator es completamente autocontenido. Todos los componentes necesarios para operar en producción están incluidos en la misma carpeta. No hay dependencias externas de otros proyectos.

|                               |                                                |
|-------------------------------|------------------------------------------------|
| ARHIAX-AgentCreator/          |                                                |
| ├─ README.md                  | ← Inicio rápido y overview                     |
| ├─ ARCHITECTURE.md            | ← Arquitectura detallada con diagramas         |
| ├─ API_REFERENCE.md           | ← Referencia completa de todos los endpoints   |
| ├─ DEPLOYMENT.md              | ← Guía de despliegue local, Linux y Kubernetes |
| ├─ SECURITY.md                | ← Política de seguridad y checklist            |
| ├─ CHANGELOG.md               | ← Historial de cambios                         |
| ├─ CONTRIBUTING.md            | ← Guía de contribución                         |
| ├─ docker-compose.yml         | ← Stack completo en un archivo                 |
| ├─ .env.example               | ← Plantilla de variables de entorno            |
| ├─ services/                  |                                                |
| │   └─ creator-api/           | ← Fábrica de agentes (FastAPI)                 |
| │       ├── main.py           |                                                |
| │       ├── requirements.txt  |                                                |
| │       └─ Dockerfile         |                                                |
| │   └─ gateway/               | ← Policy Enforcement Point (FastAPI)           |
| │   └─ aim-service/           | ← Identidad de agentes (FastAPI + SQLite)      |
| │   └─ aut-service/           | ← Autonomía A0-A4 (FastAPI + SQLite)           |
| │   └─ bbr-service/           | ← Baseline conductual (FastAPI + SQLite)       |
| │   └─ hic-service/           | ← Human-in-the-loop (FastAPI + SQLite)         |
| │   └─ evidence-store/        | ← Ledger inmutable (FastAPI + JSONL)           |
| ├─ sdk/python/                |                                                |
| │   └─ arhiax/                |                                                |
| │       ├── agent.py          | ← ARHIAXAgent + @governed_tool                 |
| │       ├── client.py         | ← GatewayClient con circuit breaker            |
| │       ├── models.py         | ← Modelos Pydantic (Credential, Decision...)   |
| │       ├── exceptions.py     | ← Excepciones ARHIAX tipadas                   |
| │       └─ __init__.py        | ← API pública del SDK                          |
| │   └─ examples/              |                                                |
| │       └─ agente_analista.py | ← Ejemplo completo de agente gobernado         |
| │   └─ setup.py               |                                                |
| ├─ runtime/                   |                                                |
| │   └─ bundles/               | ← 19 bundles de políticas OPA (Rego)           |
| │       ├── B01-B19/          | ← Bundles organizados por dominio              |
| │       └─ MANIFEST.json      |                                                |
| │   └─ atk/                   | ← Agent Trust Kernel                           |
| │   └─ c09/                   | ← Módulo C09 INTERP Bridge                     |
| │   └─ hardened/              | ← Clientes con hardening adicional             |
| │   └─ opa/                   | ← Política base main.rego                      |
| │   └─ reference-images/      | ← Implementaciones de referencia en Go         |
| ├─ chart/                     |                                                |
| │   └─ arhiax-runtime/        | ← Helm chart principal (18 templates)          |
| │   └─ arhiax-atk-standalone/ | ← Helm chart del ATK                           |

└ docs/  
└ specs/

← Documentación técnica adicional  
← Especificaciones del estándar ARHIAX

## Contacto y Soporte

---

ARHIAX AgentCreator es desarrollado y mantenido por Sinergia Consulting Group. Para soporte técnico, consultas de implementación o reportes de vulnerabilidades de seguridad, contactar directamente al equipo técnico de Sinergia Consulting Group.

Para reportar vulnerabilidades de seguridad: no abrir issues públicos. Contactar directamente al equipo con descripción, pasos para reproducir e impacto potencial.

*ARHIAX AgentCreator — Sinergia Consulting Group*